

IS 6495: Python Programming Course Project

GROUP 1:
Amiya Ranjan Sahoo,
Zachary Hancock,
Tate Harris



AIRASIA
TICKETING SYSTEM

Contents

- ✓ **Introduction**
- ✓ **Ethics and Critical Thinking**
- ✓ **Analysis & Design**
- ✓ **Infrastructure Development**
- ✓ **Business Logic Implementation**
- ✓ **System Features and User Workflow**
- ✓ **User Interface & Integration + Demo**

Introduction

Problem:

AirAsia needs a centralized system to efficiently manage flight bookings, airports, aircraft, employees, customers, and ticket sales.

Objective:

- ✓ Automate flight booking and ticket management
- ✓ Reduce errors in manual processes
- ✓ Improve customer experience and operations
- ✓ Provide admin reports for better decision-making

Ethics and Critical Thinking

Ethics and Security

✓ Customer Data Protection

Customer personal information such as name, DOB, and citizenship is securely stored in the database with controlled access.

✓ Transparent Pricing Model

The dynamic pricing engine calculates ticket prices based on flight duration, aircraft type, peak hours, and weekends — ensuring clear and fair pricing without hidden charges.

✓ No Overbooking Policy

The system respects aircraft seat capacity and prevents ticket purchases once the flight reaches its limit.

✓ User-Friendly Booking Interface

The menu-driven interface allows users to clearly see available flights, prices, and booking options before confirming a ticket.

✓ Data Integrity and Reliability

Foreign key relationships and validation rules prevent invalid or inconsistent data from entering the system.

Critical Thinking in System Design

✓ Fair Ticket Allocation

Before confirming a booking, the system verifies flight availability to avoid capacity conflicts.

✓ Logical Handling of Booking Rules

The system enforces direct-flight bookings and separates round-trip journeys into individual bookings.

✓ Error Prevention and Validation

Input validation ensures that airport codes, flight IDs, and customer records exist before processing transactions.

✓ Responsible Pricing Strategy

The dynamic pricing engine balances operational costs with fair pricing for passengers.

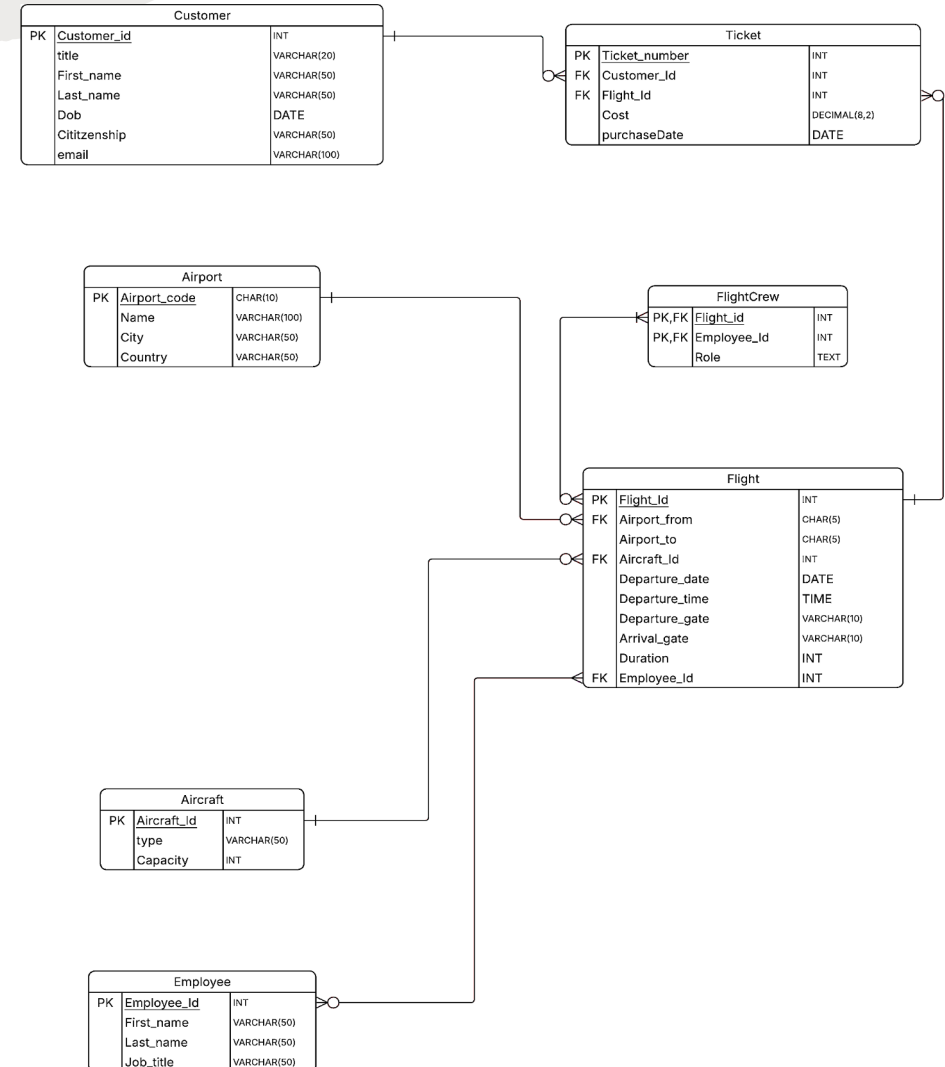
✓ Scalable System Design

The modular structure allows the system to be extended with additional features such as loyalty programs, payment systems, or analytics.



Step 1: Analysis & Design

Entity	Key Attributes
Customer	Customer_ID, Title, First_Name, Last_Name, DOB, Citizenship, Email
Airport	Airport_Code, Name, City, Country
Aircraft	Aircraft_ID, Type, Capacity
Flight	Flight_ID, Airport_From, Airport_To, Aircraft_ID, Departure_Date, Departure_Time, Departure_Gate, Arrival_Gate, Duration
Ticket	Ticket_Number, Customer_ID, Flight_ID, Cost, Purchase_Date
Employee	Employee_ID, First_Name, Last_Name, Job_Title
FlightCrew	Flight_ID, Employee_ID, Role



Step 2: *Infrastructure Development*

- ✓ **Base Classes:**
Created DBBase to manage database connections and BaseCRUD to provide reusable CRUD operations.
- ✓ **Database Schema:**
Implemented the SQLite database with **7 tables** and proper **primary and foreign key relationships**.
- ✓ **Data Population:**
Implemented **CSV import functionality** to populate the database, including **basic error handling and data validation**.
- ✓ **Object-Oriented Design:**
Developed separate Python classes for each entity (Customer, Flight, Airport, Aircraft, Employee, Ticket) to organize business logic and database operations.
- ✓ **System Integration:**
Integrated database operations with the main Python application and pricing engine to support dynamic ticket booking and flight management.

Step 3: *Business Logic Implementation*

CRUD Class Structure

- ✓ Implemented **entity-specific classes** (Customer, Flight, Airport, Ticket, Employee, etc.) to manage database operations and application logic.
- ✓ Each class handles **CRUD operations (Create, Read, Update, Delete)** for its corresponding table.
- ✓ This structure keeps **database access and business logic organized and modular**.
- ✓ By separating logic by entity, it becomes **easier to maintain, debug, and extend the application**.
- ✓ Changes made to one module have **minimal impact on other components of the system**.

Business Rules Implemented

- ✓ **Direct Flights Only:** All bookings are for direct flights between two airports.
- ✓ **Individual Bookings:** Each passenger must create a separate ticket booking.
- ✓ **Capacity Management:** Aircraft capacity limits prevent flight overbooking.
- ✓ **Data Integrity:** Foreign key constraints ensure valid relationships between customers, flights, and tickets.

Step 4: System Features and User Workflow



Customer Features

- ✓ Search available flights by departure and destination airports.
- ✓ Book tickets for direct flights and store booking details in the database.
- ✓ Cancel previously booked tickets or reschedules when required.

Pricing Engine

- ✓ Implemented a dynamic pricing function that calculates ticket prices based on factors such as flight duration, aircraft type, peak hours, and weekends.

Employee Features

- ✓ Manage employee records including pilots, flight attendants, and staff.
- ✓ Assign employees to flights through the **FlightCrew** relationship.

Flight Management

- ✓ Maintain flight schedules including aircraft, departure time, gates, and duration.
- ✓ Track passengers and crew members associated with each flight.

Step 5: *User Interface & Integration + Demo*

User Interface & Integration + Demo

- ✓ **Interactive Menu:** Implemented a menu-driven interface that allows users to perform operations such as managing customers, flights, employees, and ticket bookings.
- ✓ **System Integration:** Connected the user interface with the SQLite database and business logic classes to execute CRUD operations.
- ✓ **Error Handling:** Incorporated exception handling and input validation to ensure the system handles invalid inputs and database errors gracefully.
- ✓ **System Demonstration:** Demonstrates key workflows including flight search, ticket booking, ticket cancellation, and data retrieval.
- ✓ **Streamlit:** Explored additional functionality of moving the program into the Python Streamlit web development module. Proof of concept established and will be demod.



Thank you